AEP FROM ZERO • PART 1 • KNOW YOUR CUSTOMER (AEP)

EPISODE 15

AEP Query Service Tutorial: SQL on the Data Lake

What is Query Service in Adobe Experience Platform, and how do you run SQL on the AEP data lake? In this AEP Query Service tutorial, Bloom's team finds a dataset's table name, queries nested XDM fields with dot paths, proves a load with COUNT, and answers business questions with GROUP BY, SUM, ROUND, YEAR, ORDER BY with LIMIT and CASE. You will see why an inner join silently drops customers who never ordered, when to use a LEFT JOIN, and why Query Service is append-only:

Previously: three landmines, one question

- Last time: three landmines fixed at their layer
- A five-line root cause analysis for each
- Still open: who has never ordered at all?
- And how much did each tier really spend?
- Today: SQL over the data lake with Query Service

KEY IDEA

The landmines are cleared. Today Farhan's open question, who has never ordered at all, goes to the data lake in SQL.

Query Service: when SQL beats the UI

Reach for the screens when	Reach for Query Service when
You need one person, right now	You need counts across all history
You build one audience rule	You need sums and averages by group
You check one dataflow run	You need to join two datasets
You want the merged profile	You need to prove a load row by row

KEY IDEA

Query Service runs SQL over the data lake. It beats the screens for questions across the whole history, and it works even without the profile store.

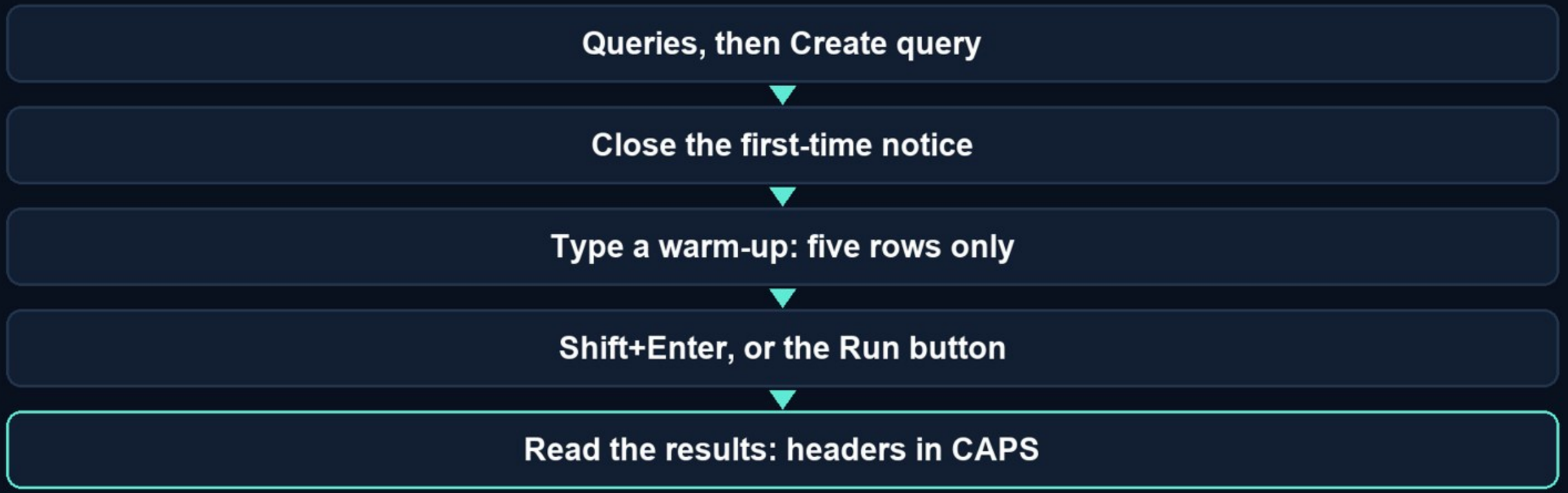
Dataset name to table name, dots for fields

What the screen shows	What SQL needs
Dataset: Bloom Customers	Table: bloom_customers
Dataset: Bloom Orders	Table: bloom_orders
Dataset: Bloom Web Events	Table: bloom_web_events
Field: email inside personalEmail	personalEmail.address

KEY IDEA

Every dataset has a table name in its right rail. Bloom Customers becomes bloom_customers, and nested XDM fields are reached with dots.

Inside the editor: Shift and Enter



KEY IDEA

Queries, then Create query, opens the editor. Shift and Enter runs a query, and the result headers come back in capitals.

Proof first: COUNT star must say 500

COUNT(*) on the trial load	What it tells you
500 on bloom_customers	The trial load is complete
Fewer than 500	The load is incomplete
1,000	The file landed twice
497 on bloom_orders	Matches episode 6: 3 rows rejected

KEY IDEA

The first query is a proof, not an insight. On the day the trial file landed, COUNT star had to say 500, or the load was incomplete or doubled.

Business questions: GROUP BY and YEAR

Business question	SQL that answers it
Customers per loyalty tier?	COUNT(*) with GROUP BY tier
Biggest tier first?	ORDER BY the count DESC
New customers by join year?	GROUP BY YEAR(join date)
The five newest customers?	ORDER BY join date DESC LIMIT 5

KEY IDEA

GROUP BY sorts rows into piles and counts each pile. YEAR groups customers by join year, and ORDER BY with LIMIT finds the newest five.

Revenue by tier: SUM, ROUND and a join

- Order prices live in bloom_orders
- Loyalty tier lives in bloom_customers
- Join each order to its customer on BloomID
- SUM the prices, ROUND to 2 decimal places
- Total revenue is not revenue per customer

KEY IDEA

Revenue lives in orders and tier lives in customers, so this answer needs a join. SUM adds, and ROUND keeps two decimal places.

CASE: who pays below list price?

- CASE: when this product, then this list price
- Compare each row's price with its list price
- Count the rows that fall below it
- Flagged: order BLM-123800 at -20.00
- The platform checks types; Bloom checks sense

KEY IDEA

CASE gives each product its list price, so SQL can flag rows sold below it. It also caught a negative price the platform let in.

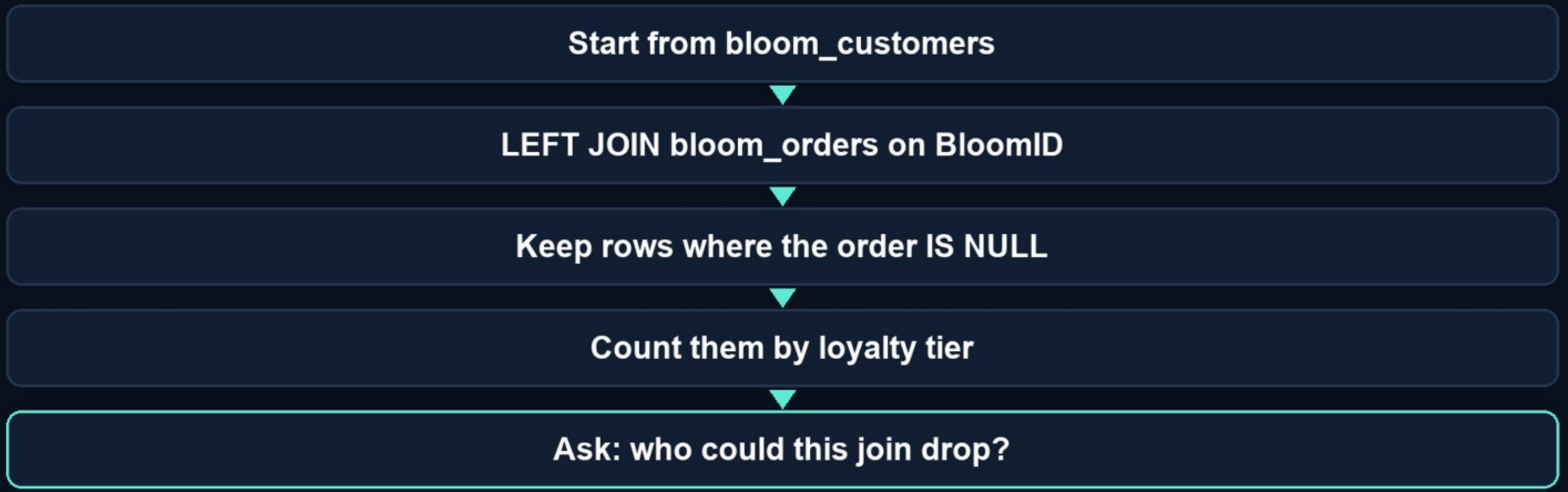
Inner join vs LEFT JOIN: the zero-order trap

Question	Inner join	LEFT JOIN
Which customers stay?	Only those with orders	Every customer
Zero-order customers	Silently dropped	Kept, orders left empty
Orders per customer	Looks higher than it is	The honest average
Can it list non-buyers?	Never	Yes: order IS NULL

KEY IDEA

An inner join keeps only customers who match an order, so non-buyers vanish and rankings shift. A LEFT JOIN keeps every customer.

Who has never ordered? The answer



KEY IDEA

A LEFT JOIN with an empty-order filter answers Farhan's question, and the tier ranking shifts once non-buyers count.

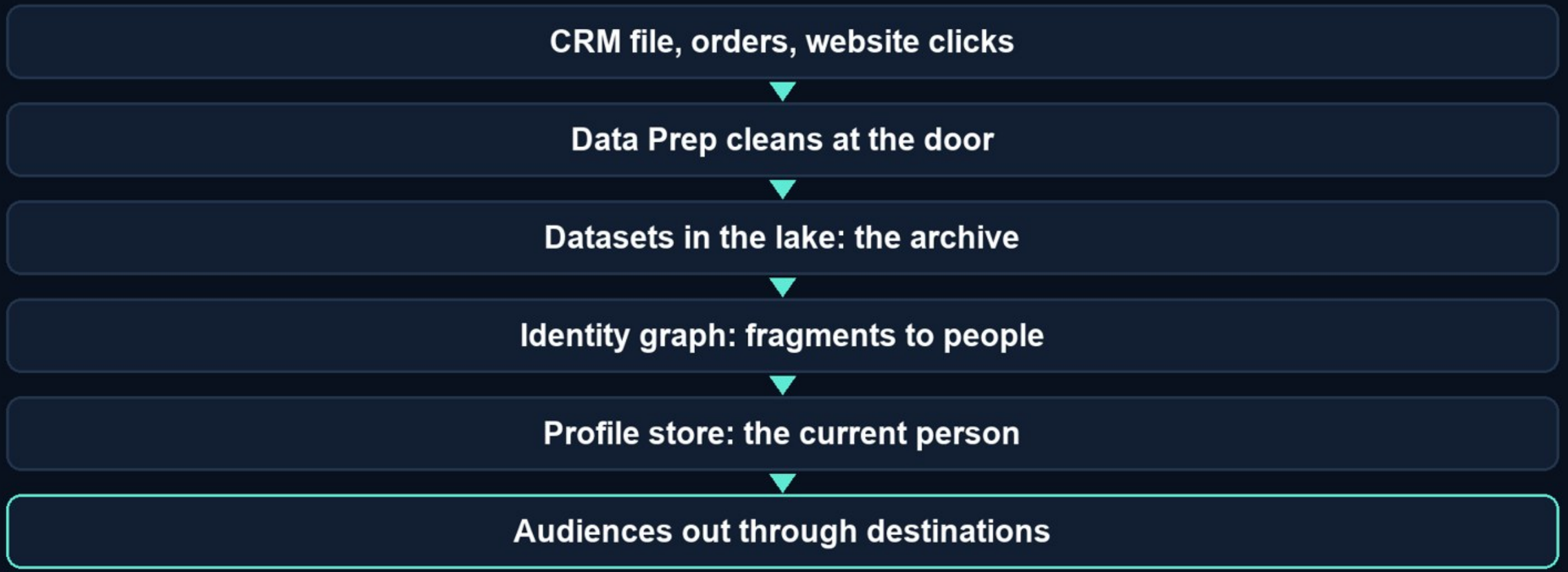
Append-only: no UPDATE and no DELETE

Command	In Query Service
SELECT	Reads anything in the data lake
INSERT INTO	Adds rows: the only way to change rows
UPDATE	Not available
DELETE	Not available

KEY IDEA

Query Service is append-only: INSERT INTO is the only command that changes a table's rows. To correct data, fix it at the source and send the corrected record in.

Act One on one whiteboard



KEY IDEA

Vikram draws Bloom's governed foundation end to end: sources, Data Prep, lake, identity, profile, audiences and destinations, with governance around it.

Interview corner: five recurring scenarios

Scenario	Answer from the build
Schema design	Orders are events; tier is profile
A failed dataflow	3 bad rows lost 497: blast radius
Identity design	Email, ECID, BloomID: one Priya
Evaluation method	React in minutes? Then streaming
Governance blocks an export	C2 on the phone; the policy held

KEY IDEA

Five AEP scenarios recur in interviews. Answer each from something you built, and lead with identity design, the most common senior question.

Recap, your question, and Divya's laptop

- Query Service: SQL over the lake, no profile needed
- Prove every load first: COUNT(*) must match
- Inner join drops non-buyers; LEFT JOIN keeps them
- Append-only: INSERT INTO, no UPDATE or DELETE
- Next: Act Two, Adobe Journey Optimizer

KEY IDEA

Count first, mind the join, and remember Query Service only appends. Next: Act Two opens with Adobe Journey Optimizer.